SOC09109 2025-6 TR2 001

Group Project

Final Delivery Report

MAP-Elites Algorithm Training

Group 277

Members:

Aswin Vazhakkoottathil Podimon (40650180)

David Weir (40654445)

Jamie Harris (40346197)

Sebastian Murray (40682339)

Robbie Black (40582304)

Kipras Tomkevicius (40681851)

Client: Simon Smith

Sponsor: Leni Le Goff

Table of Contents

1. EXECUTIVE SUMMARY 2
2. Project Management & Team Organisation 3
2.1 Team Structure and Workflow 4
2.2 Team Members and roles 4
2.3 Timeline and tasks 4
2.4 Risk Management 4
Hardware limitations 5
HPC Access 5
3. MoSCoW validation and Status 5
3.1 Must Haves 6
a. MAP-Elites Algorithm 6
b. Isaac sim integration 6
c. Parallel Computing 6
3.2 Should Have 6
3.3 Could have 7
3.4 Won't have 7
4. Technical Implementation 7
4.1 System Architecture 7
4.2 The MAP-Elites Algorithm 7
4.3 Experimental Results 8
4.4 Fitness Function 8
Appendices 8
Appendix A: Client Approval 9
Appendix B: Stakeholder Satisfaction Report 9

Appendix C: Lessons Learned 9
Appendix D: Peer Review 11
Appendix E: Gantt chart 11
Appendix F: GitHub Kanban Board 11
Appendix G: Isaac Sim Screenshots 12
Appendix H: Results 12
Appendix I: Heatmap 13
Appendix J: Pseudocode 14
Appendix K: Calendar 15

1. EXECUTIVE SUMMARY

This document serves as the formal proof of project completion for Group 277. The project delivers an implementation of the MAP-Elites algorithm in NVIDIA Isaac Sim for parallel quadruped robot controller optimisation. The project is a deliverable for SOC09109 module for client Simon Smith.

The team built five python files to share different components of the system. A neural network controller (controller.py), a behavioural archive (archive.py), an Isaac sim interface (simulation.py) and file to visualise the results(visualise.py). The algorithm evolves 160-weight neural network controller for the ant robot by running parallel episodes in Isaac sim, recording where each ant lands and how much energy it uses. It then maintains a grid of the best controller found for each region of space.

With the corrected code submitted here, we achieved these results with parallel computing:

Metric	Result
Archive coverage	100%
Best Fitness	100 (distance-weighted score)
Fitness improvement across run	Consistent increase generation 1 to 100

2. Project Management & Team Organisation

2.1 Team Structure and Workflow

The project was executed by a six-person team. Each member of the team was assigned specific technical roles with respect to their hardware access and knowledge. Work was coordinated via GitHub and Discord

2.2 Team Members and roles

2.3 Timeline and tasks

An iterative process was followed throughout the project. Tasks were broken into separate files and action. It was tracked using a Kanban board. A Gantt chart was created based on feedback from the PIR.

1. Revised Gantt Chart
   [Appendix E]

2. Kanban board
   [Appendix F]

2.4 Risk Management

Hardware limitations

During the project, hardware limitations were found. Only certain machines with specific hardware were able to run Isaac sim. One of the members of the simulation team could not physically see the robots simulated, hence after long runs of simulation the json file with genome data would be sent across teammates to verify and test again.

The mitigation strategy was an asynchronous development process. Developers would write code locally and push it to the repository's develop branch. Then the simulation

team will pull and test the code and distribute the error logs to developers. Some local changes also had to be made in certain scenarios to fit the machine. (Appendix I)

HPC Access

University High Performance Computer was initially planned for large scale training. Due to time constraints and complexity of the project, it was not achieved. The project rescoped local GPU hardware. The max generations were capped to 100. This decision is documented in main.py.

Role	Members	Responsibility
Project Manager	Aswin, Jamie	Task allocation, client and sponsor communication, documentation
Lead Developer	Robbie	main.py and archive.py
Simulation and Testing	Kipras, David	Isaac sim setup and real testing
Visualisation, Documentation	Seb	Visualisation.py, Report

3. MoSCoW validation and Status

3.1 Must Haves

Requirement	Status	Evidence
MAP-Elites algorithm in Python	Delivered	archive.py
Algorithm integrated with Isaac Sim	Delivered	simulation.py
Parallel evaluation of controllers	Delivered	controller.py
Full understanding of Isaac sim based on the Scope of the Project	Delivered	Succefully created and implemented a MAP-Elites algorithm within Isaac Sim and debugged errors.

a. MAP-Elites Algorithm

MAP-Elites is implemented across controller.py, archive.py and main.py. The Archive class sustains a 10x10 grid where each cell stores the highest fitness controller for that region in an (x,y) space. Controllers are replaced only if they improve than the existing cell occupant. The Controller class implements an 8-input, 10-hidden, 8-output MLP with 160 trainable weights.

b. Isaac sim integration

The simulation class provides an interface to Isaac Sim. It loads the simulator with new values and reads back world frame position and joint velocities after the episode.

c. Parallel Computing

Parallel evaluation is achieved through Isaac Sim's vectorised environment API. Each call to *sim.evaluate()* runs all the controls simultaneously. During testing, 40 to 500 parallel environments were confirmed running on an NVIDIA RTX 3050 Ti. See Appendix F for screenshots.

3.2 Should Have

Requirement	Status	Evidence
Algorithm Documentation	Delivered	README.md and inline comments
Psuedocode	Delivered	Appendix I
Code comments	Delivered	All five files contain docstrings
Research Notes	Delivered	Notes folder in the Repository

3.3 Could have

Requirement	Status	Reason
Physical robot implementation	Not delivered	Time constraints
Algorithm for different test conditions	Not delivered	Time constraints

3.4 Won't have

Requirement	Status	Reason
Running physical quadruped robot	Not delivered	Out of scope per PIR
Jumping quadruped robot	Not delivered	Out of scope per PIR

4. Technical Implementation

4.1 System Architecture

The project is structured to have five python files.

File	Purpose	Owner
controller.py	Neural network brain, 160 weights, mutation, crossover	Jamie
archive.py	MAP-Elites grid, insert/sample logic, save/load	Robbie
simulation.py	Isaac Sim interface mock and real modes, parallel evaluation	David, Kipras
main.py	Evolutionary loop, ties all modules together	Robbie

visualisation.py	archive heatmap and metrics graphs from CSV	Sebastian

4.2 The MAP-Elites Algorithm

The MAP-Elites is quality-diversity algorithm that maintains a collection of high-performing solutions across a space of behaviours. MAP-Elites seeks to fill a behavioural map with the best solution found for each behavioural niche.

The algorithm as implemented:

- **Initialisation:** generate POPULATION_SIZE random controller genomes
- **Evaluation:** run each controller in Isaac Sim for one episode (200 timesteps)
- **Description:** extract final (x,y) position as the behaviour descriptor
- **Score:** compute fitness as a distance weighted energy score
- **Archive:** insert into 10x10 grid. Replace only if there is an improvement
- **Selection:** sample controllers from the archive for the next generation
- **Variation:** apply gaussian mutation (strength 0.25) and blend crossover
- Repeat for *MAX_GENERATIONS* iterations

4.3 Experimental Results

Three experimental configurations were run. The key results are summarised below:

Running 100 environments running the sim for over 100 generations would sometimes cause to the sim hanging, leading to a crash. We then limited our generations to 100 and environment number to 50 –100. (Figure G1)

Initially the calculation for fitness was based on energy, however this led to the robots prioritizing energy too much causing them to simply stand still and use the least energy possible (Appendix G). This led to positions being very close to (0,0) (Figure G2) as seen in the screenshot. The fitness formula was revised to weigh distance more heavily. (Figure G3)

4.4 Fitness Function

The fitness function was developed iteratively during testing. The team made changes to the prioritisation of the distance parameter through iterations.

The final real-sim fitness formula rewards distance travelled while penalising energy expenditure:

*fitness = 10 * distance – 0.1 * avg_energy*

Figure G3 shows an earlier iteration of the formula for context

Appendices

Appendix A: Client Approval

[screenshot of final email from simon]

Appendix B: Stakeholder Satisfaction Report

2-3 sentence objective summary of the feedback received from Simon during the project or at the final handover.

Appendix C: Lessons Learned

1. **Unforeseen absences and late admission**

Problem: The project faced early disruption when the original project manager required authorised absence during the initiation phase. Separately, a sixth team member (Robbie) joined in week 5 of the project.

Improvement Implemented: A team member stepped up as interim PM (Jamie), taking ownership of the PIR and client meetings. Robbie was introduced to the system and assignments were redistributed accordingly and communicated via Discord.

Reflection: In future projects, a documented role succession plan should be established at the outset so that any absence does not disrupt the workflow. Keeping a document underlining the project structure and details could be useful if addition of members is needed in the team. The team adapted well under the circumstances.

2. **Hardware constraints and asynchronous development**

Problem: The team lacked consistent access to machines with the required GPU specifications to run Isaac Sim natively. Developers had to write code without having access to a simulator to test their code.

Improvement Implemented: A *USE_MOCK = True* toggle and *_run_mock()* function was introduced in *simulation.py* and *main.py,* allowing developers without Isaac Sim access to run their code and look for errors without using the simulator. Code was developed locally and pushed to github. The simulation team would test the code and give feedback to the developers.

Reflection: The mock system proved to be a quick fix that enabled faster coding. However, there were a lot of bugs as it was not written for the simulator. In future, establishing a shared testing environment (such as a remote development server or cloud GPU instance) earlier in the project would close this gap.

3. **Inability to Access High-Performance Computing (HPC)**

Problem: The team initially planned to use the university's HPC cluster to run large-scale training. This proved unachievable due to a combination of time constraints and insufficient familiarity with Slurm job scheduling and Apptainer containerisation at the project's outset.

Improvement Implemented: The project was rescoped to run entirely on local GPU hardware. Parameters were adjusted to fit available resources: *MAX_GENERATIONS = 100 , POPULATION_SIZE = 5 , and device = torch.device("cpu") in main.py*.

Reflection: The team underestimated the initial setup overhead involved. In a future project with similar requirements, we would allocate a dedicated sprint early on to environment setup and access verification, independent of algorithm development. It would have been preferable to make this decision earlier rather than under time pressure.

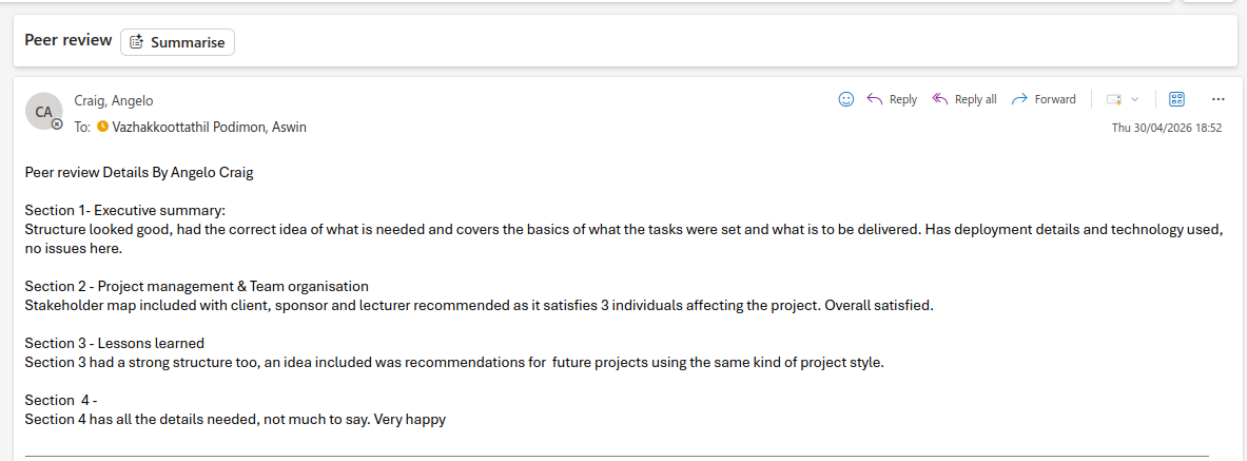
4. **Conflicting Module Deadlines and Time Constraints**

Problem: Concurrent coursework deadlines from other modules significantly reduced available working hours across the team. This caused initial task estimates on the Gantt chart to slip and created periods where progress stalled.

Improvement Implemented: The team decided to solely focus on delivering Must-have requirements first. The team also created a calendar in week 4 to figure out which days we could work together. [Appendix K]

Reflection: The initial Gantt chart was built on optimistic availability estimates that did not account for other module deadlines. The change to a tighter agile cycle was effective once made but applying it from the very beginning rather than a reactive measure would have prevented the initial pressure which involved meetings with a lot of the members missing due to conflicts.

Appendix D: Peer Review



Peer review Summarise

CA Craig, Angelo
To: Vazhakkoottathil Podimon, Aswin

Reply Reply all Forward

Thu 30/04/2026 18:52

Peer review Details By Angelo Craig

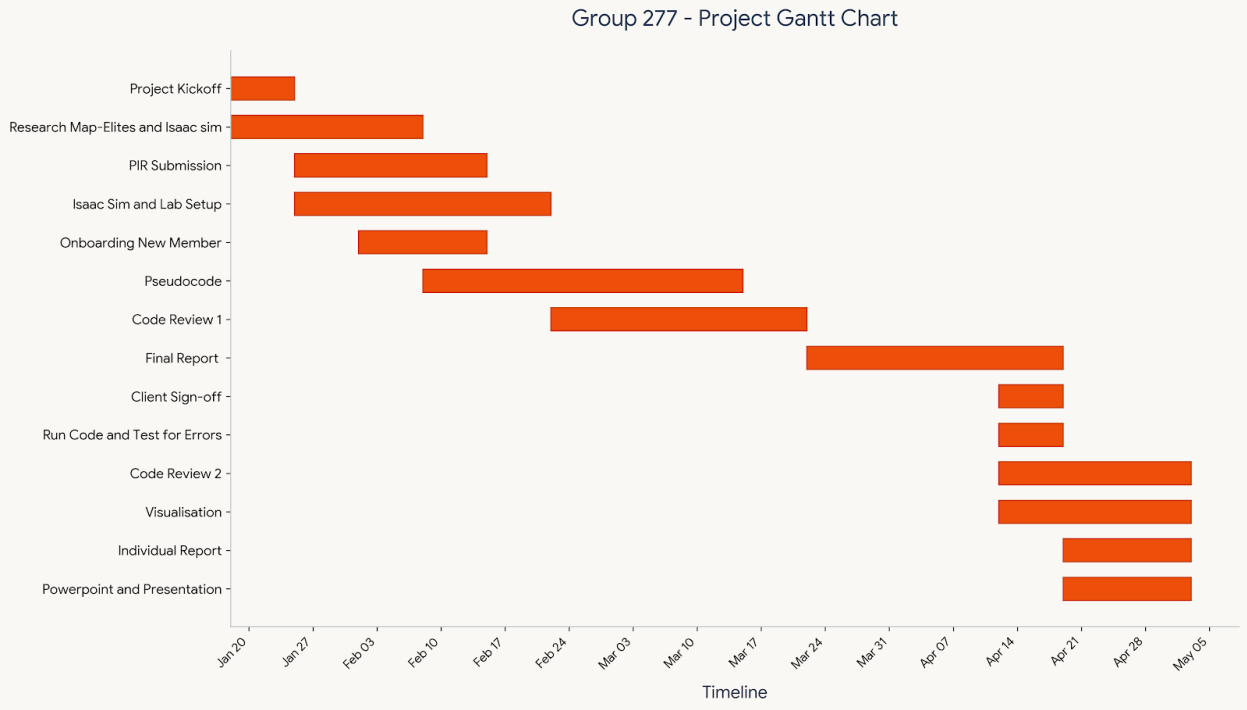
Section 1- Executive summary:
Structure looked good, had the correct idea of what is needed and covers the basics of what the tasks were set and what is to be delivered. Has deployment details and technology used, no issues here.

Section 2 - Project management & Team organisation
Stakeholder map included with client, sponsor and lecturer recommended as it satisfies 3 individuals affecting the project. Overall satisfied.

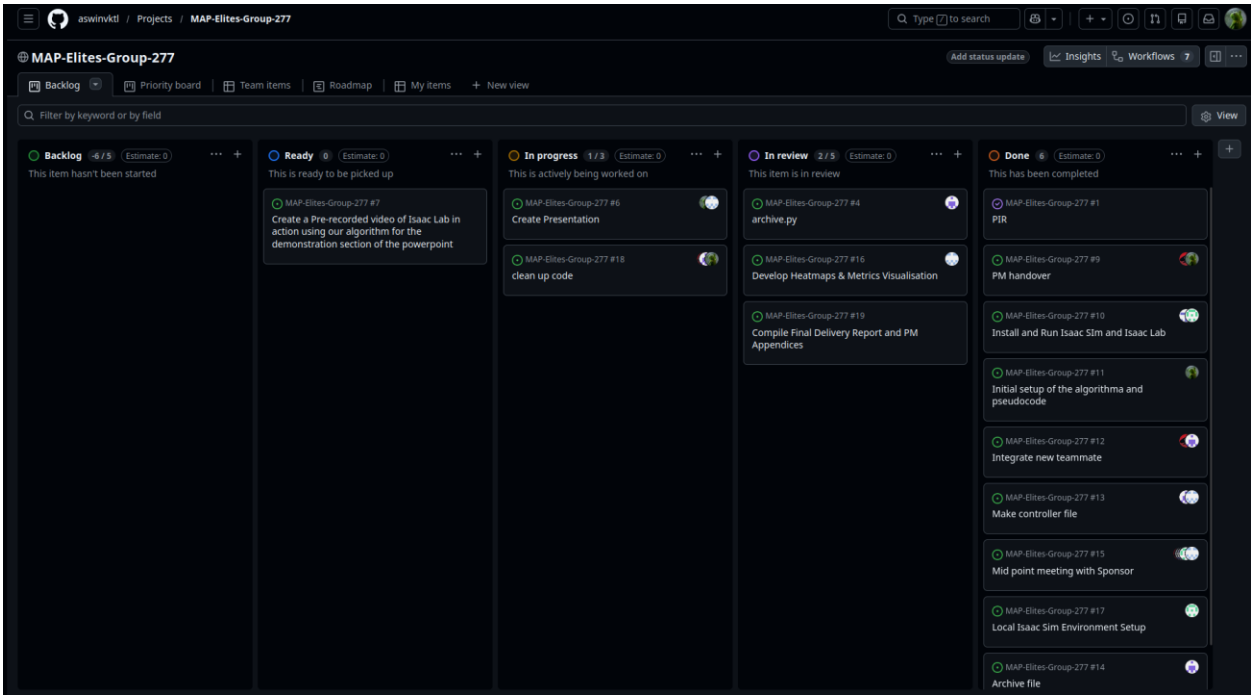
Section 3 - Lessons learned
Section 3 had a strong structure too, an idea included was recommendations for future projects using the same kind of project style.

Section 4 -
Section 4 has all the details needed, not much to say. Very happy

Appendix E: Gantt chart

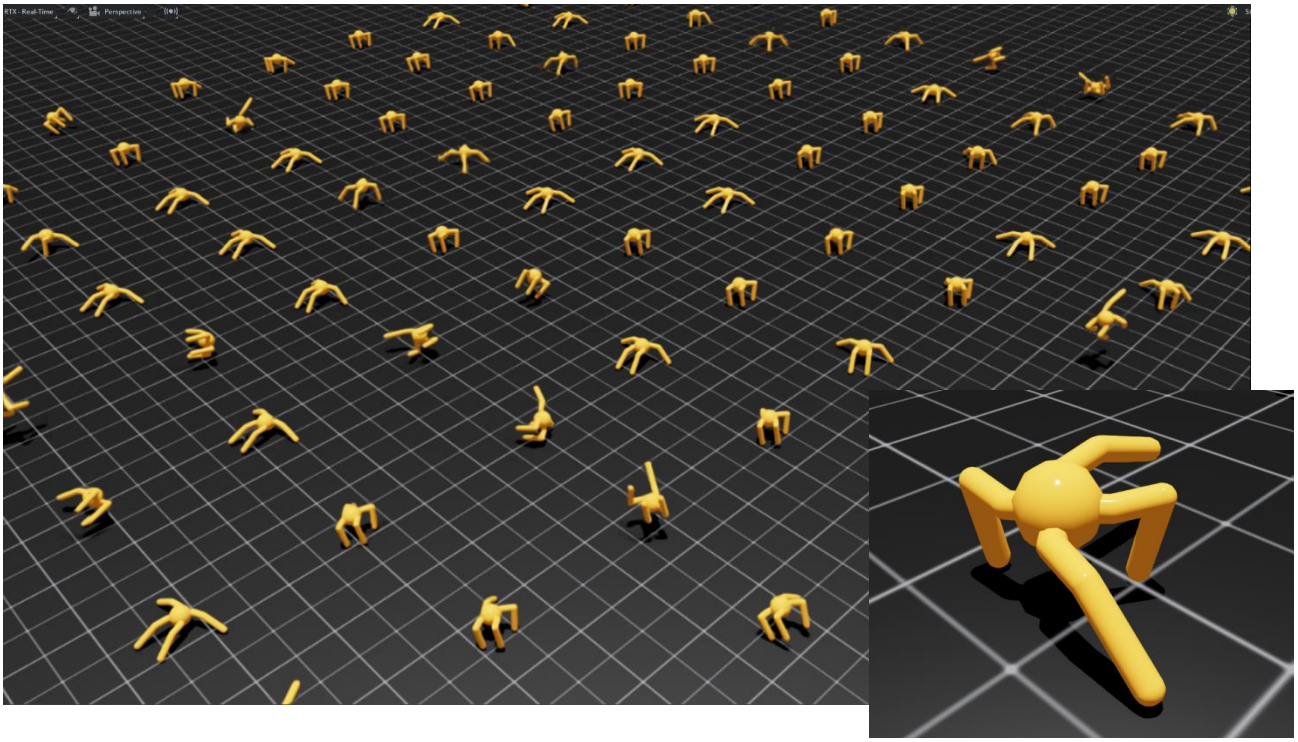


Appendix F: GitHub Kanban Board



Edinburgh Napier UNIVERSITY

Appendix G: Isaac Sim Screenshots



Appendix H: Results

```
        avg_energy = total_energy / self.episode_length
        distance = (final_x**2 + final_y**2)**0.5
        fitness = distance - 0.1 * avg_energy
        print(fitness)

        print(f"  [SIM] Controller {idx + 1}: energy={avg_energy:.4f}, pos=({final_x:.2f}, {final_y:.2f})")
        results.append((fitness, final_x, final_y))

    return results
```

Figure G1: Isaac Sim crash dialog encountered during 100-environment runs

Edinburgh Napier
UNIVERSITY

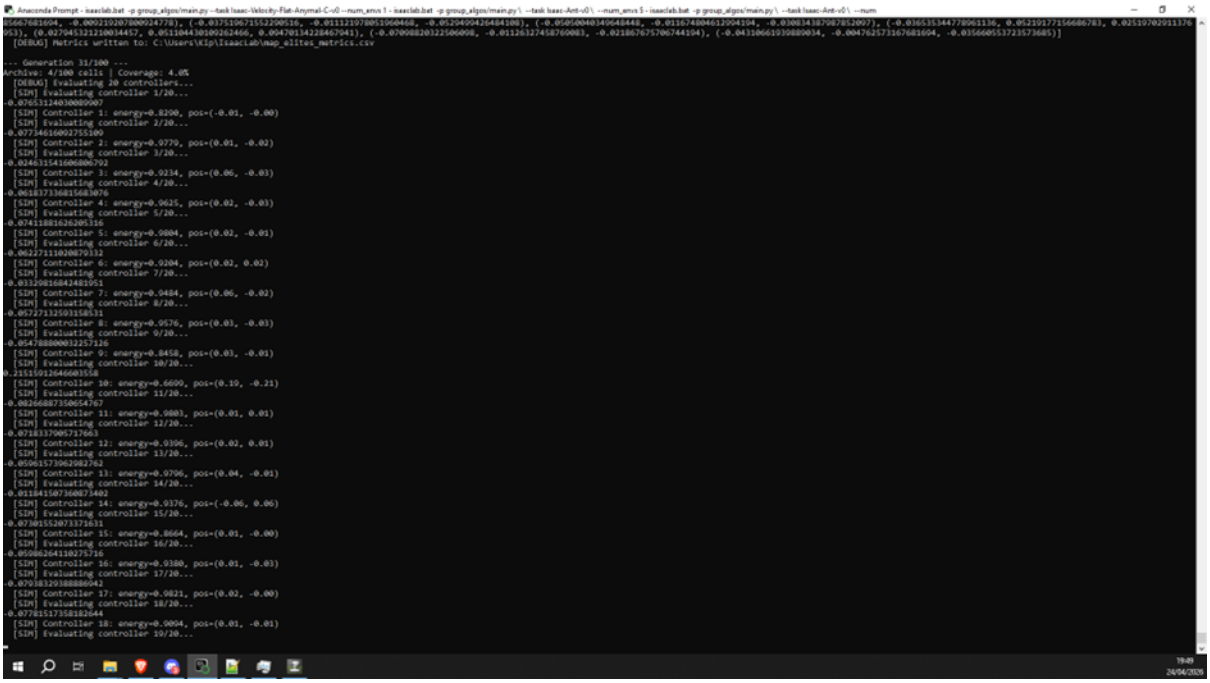


Figure G2: The output from initial configuration

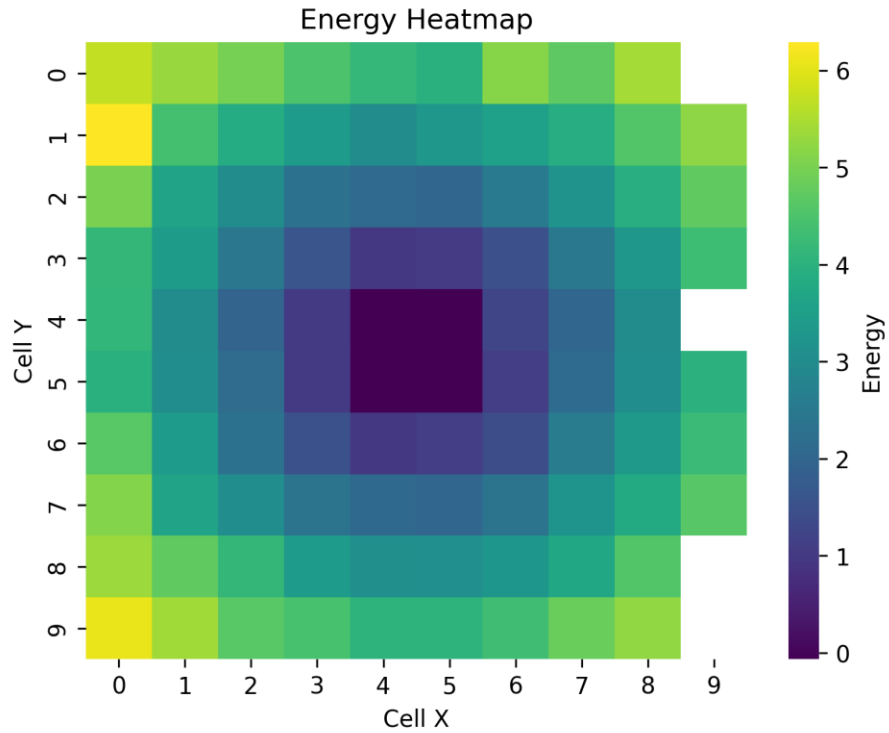
```
distance = (final_x**2 + final_y**2)**0.5
fitness =  distance - 0.1 * avg_energy
```

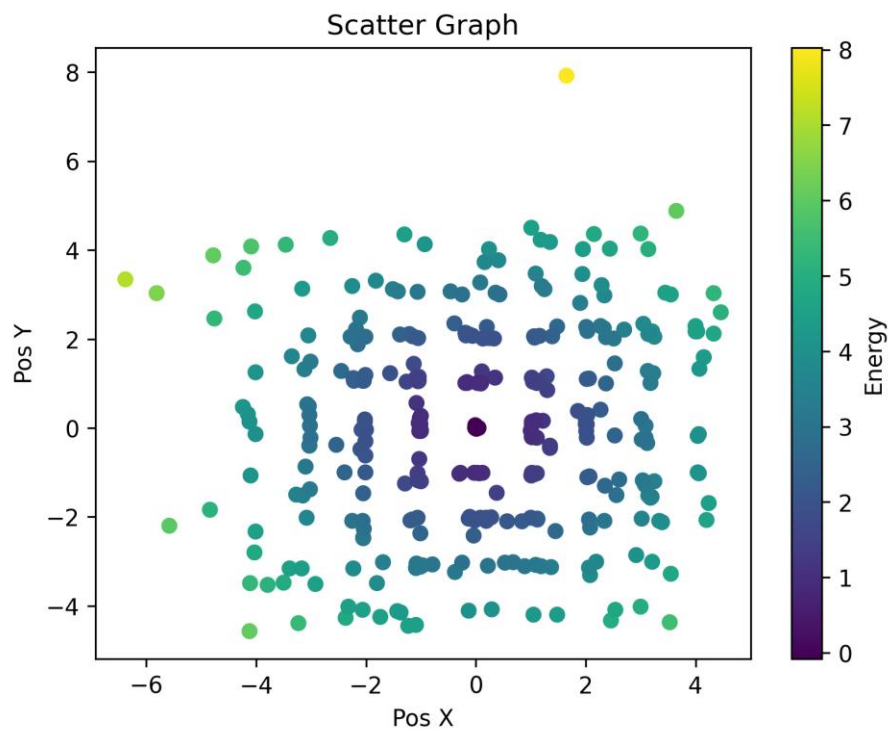
Figure G3: fitness formula prioritizing distance travelled more over average energy expenditure (Old iteration)

(need 100% csv and screenshot of sim having 100% coverage here)

Edinburgh Napier
UNIVERSITY

Appendix I: Heatmap





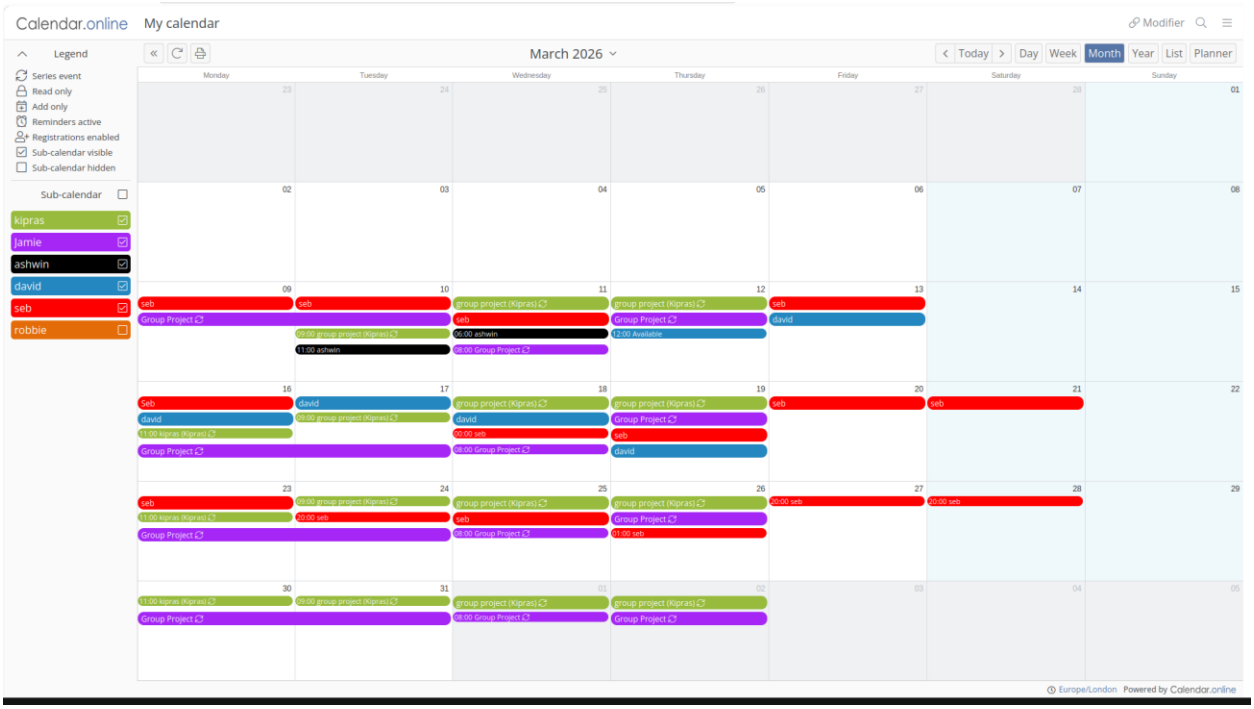
Edinburgh Napier UNIVERSITY

Appendix J: Pseudocode

```python
# --- MAIN PROGRAM ---
def main():
    generation = 0

    # Load old results if they exist
    load_archive()

    # --- MAIN LOOP ---
    while generation < max_generations:
        print(f"\n--- Generation {generation + 1}/{max_generations} ---")
        coverage = get_archive_coverage()
        print(f"Archive size: {len(archive)}/100 | Coverage: {coverage:.1%}")

        # Test each ant in this generation
        for pop_idx in range(population_size):
            # Make a new brain
            if random.random() < 0.5 and len(archive) >= 2:
                # Mix two good brains
                parent1, parent2 = sample_two_parents()
                genome = crossover(parent1, parent2)
                genome = mutate(genome)
            else:
                # Change an existing brain
                parent = sample_parent()
                if parent is None:
                    policy = create_policy()
                    genome = policy.get_weights_as_vector()
                else:
                    genome = mutate(parent)



            # Now we just make up numbers to test the algorithm
            descriptor = compute_behavior_descriptor()
            fitness = compute_fitness()


            cell = get_cell(descriptor)

            # Save to archive if it's the best so far for this cell
            if cell not in archive or fitness > archive[cell]["fitness"]:
                archive[cell] = {
                    "genome": genome.clone().cpu(),
                    "fitness": fitness,
                    "descriptor": descriptor,
                }
                print(f"New elite in cell {cell} | fitness: {fitness:.3f}")

        # Save progress
        best_fitness = max([data["fitness"] for data in archive.values()]) if archive else 0
        log_metrics(generation + 1, len(archive), get_archive_coverage(), best_fitness)
```


Edinburgh Napier
UNIVERSITY

Appendix K: Calendar



Edinburgh Napier UNIVERSITY